

Vår 24, versjon 2

Del I

Kompilering, linking, kjøring

1. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☐ Man kan ta en kjørefil (*executable*) og kjøre den på et hvilket som helst operativsystem.
- ☐ Når man kjører et compilert program vil man ikke få kjøretidsfeil.
- ☐ Man kan ta en kjørefil (*executable*) og kjøre den på hvilken som helst datamaskin.
- ☒ **Før man kan kjøre et C++ program må man kompilere kildekoden.**

Typekonvertering

2. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☒ **Man kan alltid gjøre eksplisitt typekonvertering heller enn implisitt typekonvertering.**
- ☐ Man kan konvertere en float til en int uten tap av presisjon.
- ☐ Man kan alltid konvertere en unsigned char til en char uten tap av presisjon.
- ☒ **Konvertering fra en int til en float kan gjøres implisitt.**

Definisjoner og deklarasjoner

3. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☒ **En klasse kan deklarerer flere ganger.**
- ☒ **En funksjon kan deklarerer flere ganger.**
- ☐ En medlemsfunksjon kan defineres flere ganger.
- ☐ En funksjon kan defineres flere ganger, men bare én gang i hver .cpp-fil.

Skop

4. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☐ Man kan ikke definere en klasse i det globale skopet.
- ☐ Man kan ikke definere en klasse i skopet til en funksjon.
- ☒ **Man kan deklarerer nye medlemsfunksjoner til en klasse etter at klassen er deklarerert.**
- ☒ **Man kan definere en medlemsfunksjon både i og utenfor klassedefinisjonen.**

RAII

5. [5]

Se på koden under.

```

1  int* foo (int a) {
2      int *ptr = new int(a);
3      return ptr;
4  }
5
6  void bar(int b) {
7      int* ptr = foo(b);
8      std::vector<int> vec;
9      vec.push_back(b);
10 }
11
12 int main() {
13     bar(5);
14     return 0;
15 }

```

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☒ **Koden inneholder en minnelekkasje.**
- ☐ På linje 14 vil minnet som ptr peker til være frigjort.
- ☒ **På linje 14 vil ressursene allokert til vec være frigjort.**
- ☐ Det allokeres ingen ressurser til vec som må frigjøres.

Klassemedlemmer

6. [5] Hvilke (en eller flere) av disse kan være medlemmer i en klasse?

- ☒ **Variabler**
- ☒ **Variabler merket som constexpr**
- ☒ **Statiske funksjoner**
- ☒ **Funksjoner**

Rent virtuelle destruktører

7. [5]

Hvilken funksjon (en eller flere) har en rent virtuell (*pure virtual*) destruktør?

- ☒ **Tvinge arvende klasser til å implementere egne destruktører.**
- ☐ Forsikre at destruktøren ikke kalles for å bedre ytelsen.
- ☐ Sikre at man ikke kan initialisere et objekt av baseklassen.
- ☐ Sikre at en klasse ikke kan arves fra.

Unntakshåndtering

8. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☐ Alle ressurser blir automatisk frigjort når et unntak kastes.
- ☒ **C++ splitter unntak i detektering, via en try-blokk, og håndtering, via catch-blokker.**
- ☐ Hvis du bruker at()-funksjonen for å indeksere en `std::vector`, men indeksen er utenfor rekkevidde, kan du fange dette med en `std::runtime_error`.
- ☒ **Unntakshåndtering er et eksempel på flytkontroll.**

Pekere

9. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☐ Hvis man vil finne minneadressen til en verdi kan man bruke `*`-operatoren.
- ☐ Hvis man vil lese verdien som en peker peker til kan man bruke `&`-operatoren.
- ☒ Hvis man vil endre verdien som en peker peker til kan man bruke `*`-operatoren.
- ☒ Hvis man vil endre verdien som en peker peker til kan man bruke `[]`-operatoren.

Kopiering

10. [5]

Se på koden under. Sjå på koden under.

```
1 double* euler1 = new double{0.577};
2 double* euler2 = new double{*euler1};
3 *euler1 = 2.718;
4 *euler2 = 0.596;
```

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☒ Koden er et eksempel på dyp kopiering (*deep copy*).
- ☐ Koden er et eksempel på grunn kopiering (*shallow copy*).
- ☐ Etter linje 4 peker `euler1` og `euler2` på samme verdier.
- ☐ Etter linje 4 vil `euler2` være en peker til en peker.

Tilgangsnivå

11. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte?

- ☒ Abstrakte klasser kan ha medlemmer som er `protected`.
- ☐ Et klassemedlem med tilgangsnivået `protected` kan bare aksesseres i filen den er deklartert.
- ☐ En funksjon med tilgangsnivået `protected` kan ikke overlastes.
- ☐ En funksjon med tilgangsnivået `protected` kan aksessere klassemedlemmer med tilgangsnivået `private` i foreldreklassen.

Beholdere

12. [5]

Hvilke (en eller flere) av følgende utsagn er korrekte om `std::vector`?

- ☒ Verdien returnert av `size()` overstiger aldri verdien returnert av `capacity()`.
- ☐ `push_back()` endrer alltid både verdien returnert av `size()` og verdien returnert av `capacity()`.
- ☐ `resize()` endrer alltid både verdien returnert av `size()` og verdien returnert av `capacity()`.
- ☒ `emplace_back()` kan endre verdien returnert av `capacity()`.

Del II

Testing

13. [5]

Forklar kort forskjellen på enhetstesting (*unit testing*), systemtesting (*system testing*) og regresjonstesting (*regression testing*)

Solution:

Enhetstesting Test av mindre deler av et program, som for eksempel en funksjon eller en klasse. Hvis en enhetstest returnerer en feil, vet man at feilen er i den enheten man tester. Det er dermed lettere å finne feilen.

Systemtesting Test av hele systemet. Hvis en systemtest returnerer en feil, vet man kun at feilen ligger et eller annet sted i programmet. Det er dermed vanskeligere å finne feilen.

Regresjonstesting Omfatter både enhetstesting og systemtesting for å sjekke at endringer man har gjort i et program ikke har introdusert feil.

(23.3, s.992-993 i læreboka)

Variabler

14. [5] Hva skjer når man bruker auto nøkkelordet til å deklarere en variabel og når kan det være nyttig å bruke det?

Solution: Når man deklarerer en variabel med auto nøkkelordet vil variabelen få typen til verdien den tilordnes. Dette kan være nytting når typen til variabelen har et langt og komplisert navn, for eksempel hvis man deklarerer en iterator variabel. (Andre rimelige bruksområder godtas også.)

(21.2, s.760 i læreboka)

Funksjoner

15. [5]

Se på koden under.

```
void scale(double& number, double& factor) {  
    number = number * factor;  
}
```

Hvordan blir argumentene gitt til funksjonen `scale`? Syns du dette er et godt valg eller ville du gjort det på en annen måte? Begrunn svaret kort.

Solution: Både argumentet `number` og argumentet `factor` gis til funksjonen ved hjelp av en referanse (*pass-by-reference*). For `number` er en referanse nødvendig fordi funksjonen endrer verdien til argumentet. For `factor` hadde det vært bedre å gi argumentet som en verdi (*pass-by-value*) siden argumentet ikke endres og kun er én flyttsverdi.

(8.5.3-8.5.6, s. 275 - 284 i læreboka)

Referanser

16. [5]

Forklar kort hvorfor referanser er nyttige.

Solution:

- Man kan lage funksjoner med flere returverdier.
- Man kan lage funksjoner som endrer et argument.
- Man kan unngå unødvendig kopiering.
- Man kan lage en variabel som kun kan leses.

(Forelesning 6)

Beholdere

17. [5]

Hvorfor gir det mening at `std::map` ikke støtter `std::push_back()`?

Solution: Elementene i et `std::map` er ordnet basert på nøkkelen i (nøkkel,verdi) paret, så det gir mening at det ikke er mulig å plassere et element bakerst i sekvensen, som er funksjonaliteten til `std::push_back()`.

(21.6, s.776-788 i læreboka)

Lagring og pekere

18. [5]

Se på koden under. Sjå på koden under.

```
1  #include <iostream>
2  #include <vector>
3  #include <array>
4
5  bool print = true;
6
7  int f(std::vector<int>& vec, std::map<int, int> map, int n) {
8      return vec[n] - map[n];
9  }
10
11 int main(void) {
12     std::vector<int> a = {1, 3, 18};
13     int b[] = {4, 9, 2};
14
15     int diffN = f(a, b, 1);
16
17     int* ptr = new int(1);
18     *ptr = diffN;
19 }
```

```

20     if (print) {
21         std::cout << *ptr << std::endl;
22     }
23
24     delete ptr;
25
26     return 0;
27 }

```

I hvilket minneområde blir de ulike variablene og tilhørende data lagret?

Solution:

- print: statisk lager
- vec og a: heap (data) og stakk (kontrollinformasjon)
- arr, n, b og diffN: stakk
- ptr: heap (data ptr peker på) og stakk (ptr)

(17.4, s.591-592 i læreboka, forelesning 10)

Klasser

19. [10]

Se på koden under.

```

1  #include <iostream>
2
3  class A {
4  private:
5      int x;
6      int y;
7  public:
8      virtual void foo() = 0;
9      virtual void bar() const;
10
11      A(int x_, int y_) : x(x_), y(y_) {}
12 };
13
14 class B : public A {
15 public:
16     B(int x_, int y_) : A(x_, y_) {}
17
18     void bar() const override {
19         std::cout << "bar" << std::endl;
20     }
21 };
22
23 class C : public B {
24 public:

```

```

25         C(int x_, int y_) : B(x_, y_) {}
26
27         void foo() override {
28             std::cout << "foo" << std::endl;
29         }
30     };
31
32     int main(void) {
33         B b(3, 4);
34         C c(5, 6);
35
36         c.foo();
37         b.bar();
38
39         return 0;
40     }

```

(a) (5 points) Hvorfor kompilerer ikke koden? Forklar kort.

Solution: Koden kompilerer ikke fordi vi prøver å lage et objekt av den abstrakte klassen B, noe som ikke er mulig.
(14.2.1, 14.3, s.495-496, 504-513 i læreboka)

(b) (5 points) Hva må gjøres for å løse problemet gitt at du ikke kan endre noe i main?

Solution: Den rent virtuelle funksjonen foo må overstyres i klassen B.
(14.2.1, 14.3, s.495-496, 504-513 i læreboka)

Header-filer

20. [5]

Under er kode i tre filer `f1.h`, `f2.h` og `main.cpp`. Hva er problemet og hva ville du gjort for å fikse det uten å fjerne eller flytte på koden som står der?

```

// f1.h
double get_pi() {
    return 3.14;
}

// f2.h
#include "f1.h"

double area(double r) {
    double pi = get_pi();
    return r * pi * pi;
}

// main.cpp
#include "f1.h"
#include "f2.h"
#include <iostream>

int main() {

```

```
std::cout << area(5.0) << std::endl;  
return 0;  
}
```

Solution: Problemet er at funksjonen `get_pi()` blir definert to ganger når `f1.h` inkluderes i både `f2.h` og `main.cpp`. For å løse dette må `#pragma once` stå øverst i `f1.h`. (Det er god skikk og skrive `#pragma once` øverst i alle header-filer, så det burde også stå øverst i `f2.h`, men det vil compilere dersom det kun legges til i `f1.h`)
(forelesning 5)

Templates

21. [10]

Se på koden under.

```
class Point {  
    int x = 0;  
    int y = 0;  
public:  
    int getX() {  
        return x;  
    }  
  
    int getY() {  
        return y;  
    }  
};
```

- (a) (5 points) Endre Point-klassen til en template-klasse slik at vi kan ha ulike datatyper for koordinatene x og y. Du kan velge å skrive kode eller fortelle hvilke linjer du vil endre og hvordan.

Solution:

```
template<typename T>  
class Point {  
    T x = 0;  
    T y = 0;  
public:  
    T getX() {  
        return x;  
    }  
  
    T getY() {  
        return y;  
    }  
};
```

(19.3-19.3.4, s.678-686 i læreboka, forelesning 13)

- (b) (5 points) Hva kan være fordelen ved å ha Point som en template-klasse?

Solution: Når Point er en template-klasse kan vi gjenbruke koden hvis vi har punkter som spesifiseres av ulike typer, e.g., int, float.

(19.3-19.3.4, s.678-686 i læreboka, forelesning 13)

I/O

22. [5]

Forklar kort forskjellen mellom virkemåten til >>-operatoren og get_line().

Solution: >>-operatoren leser til objekter av en bestemt type og vil lese helt fram til den finner noe som ikke stemmer med typen til objektet den leser til, e.g., hvis vi leser til en `int` vil man lese fram til man finner noe annet enn et tall. `get_line()` vil lese hele linjer med input på en gang.
(11.5, s.395-396 i læreboka)